# Final Report: Kaggle SPR X-Ray Gender Prediction Challenge

- **Challenge Name: "SPR X-Ray Gender Prediction Challenge" by Kaggle**
- **Challenge Access: Link.**
- **Task: Predicting the gender of the patient from Chest X-ray images.**
- **Date: August 6th, 2025**
- **Jupyter Notebook Access: Git repository.**

## 1. Project Overview and Context

This project was initially developed around two years ago (**July 2023**) as part of the SPR X-Ray Gender Prediction Kaggle Challenge during the BumbleKite Summer School at ETH Zurich. The task involved training a deep learning model to predict patient gender from chest X-ray images, with over 22,000 images provided.

At the time, I was just starting to learn about Machine Learning. The project was instrumental in helping me build early intuition about working with medical imaging data, setting up CNN models, and navigating real-world training constraints on limited hardware (Apple M1 chip). Since then, I've gained significant experience through research in medical AI, rigorous ML model development, and writing production-quality codebases.

In **August 2025**, while reviewing this project as part of organizing my GitHub portfolio, I realized that the original notebook version contained many issues typical of beginner work: messy structure, improper validation techniques, and some flawed assumptions about model evaluation. Thus, I decided to **revisit this project** to bring it up to standard and **better reflect my current abilities**.

## 2. Revisited Implementation: What Was Updated and Why

In this updated version, I aimed to refine the entire pipeline, from preprocessing and modeling to evaluation, with clarity, reproducibility, and best practices in mind. Key changes include:

- Data Handling & Splitting
    - Splitting the training data into train and validation *before* tuning or training to avoid data leakage
    - Ensuring proper use of the validation set for hyperparameter tuning
    - Clear separation between training, validation, and test sets
- Training Pipeline & Code Structure
    - Using reproducible, modular code (e.g., reusable initialization functions and introducing better naming conventions)
    - Refactoring training loops to improve clarity and maintainability (e.g, memory-efficient operations)
- Model Architecture & Tuning
    - Improved model architecture with dropout for regularization
    - Learning rate tuning and change of the number of epochs for better convergence
- Evaluation & Analysis
    - Adding proper evaluation metrics and learning curve analysis.

This revision represents both a cleaned-up version of the project and a snapshot of how far I've come since I started learning Machine Learning. I'm including both versions in the GitHub repository to demonstrate my

growth, my commitment to improving my work, and my ability to critically assess and build upon past experiences.

## 3. Model Tuning

While several improvements were made to the original implementation, this section focuses specifically on the model tuning process. Tuning played a key role in improving the model's generalization and convergence behavior, and best illustrates the reasoning behind the final performance achieved. For a complete view of all code changes, the final version of the notebook is available in the root directory of the Git repository, with the original draft preserved in the archive/ folder for transparency and reflection on progress.

As an initial step in tuning, I trained the model with a Learning Rate (LR) of 0.01 over 50 epochs. The resulting learning curve (see Figure 1) showed a large gap between the training and validation lines, notably after around epoch 15, with training loss steadily decreasing while validation loss plateaued and even slightly increased over time. This clear divergence indicated that the model was overfitting on the training data, learning patterns too specific to the training set, and failing to generalize well to unseen data. This insight prompted further adjustments to improve generalization.

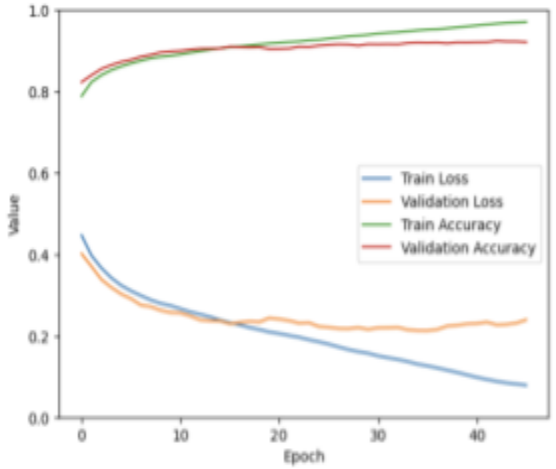


*Figure 1. Learning curve (loss and accuracy) with LR=0.01 over 50 epochs showing overfitting behavior.*

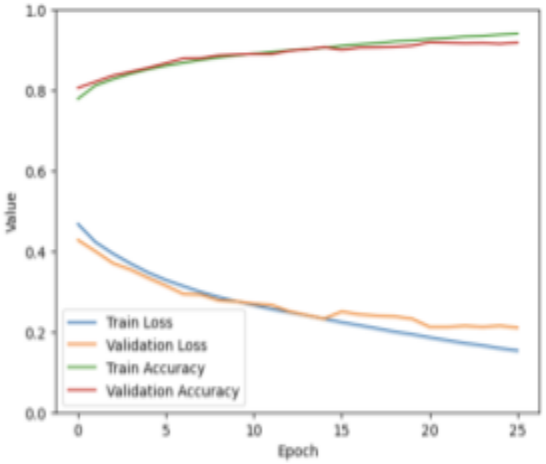


*Figure 2. Learning curve with LR=0.01 over 30 epochs.*

In the next iteration, the learning rate remained unchanged (0.01), but the number of epochs was reduced to 30 to mitigate the overfitting observed earlier (see Figure 2). This adjustment led to a modest improvement: while the training loss continued to decrease, the gap between training and validation loss narrowed slightly. However, signs of overfitting remained, as the validation loss plateaued and began to fluctuate after epoch 15, suggesting limited generalization despite shorter training. Despite this, the model achieved a validation accuracy of 91.36%, with a precision of 86.96%, a recall of 93.44%, and an F1-score of 90.08%, indicating reasonably strong classification performance.

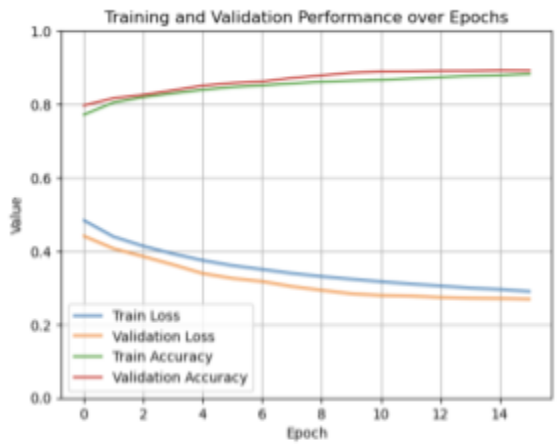


*Figure 3. Learning curve (loss and accuracy) with LR=0.01 and 20 epochs, incorporating dropout layers to reduce overfitting.*

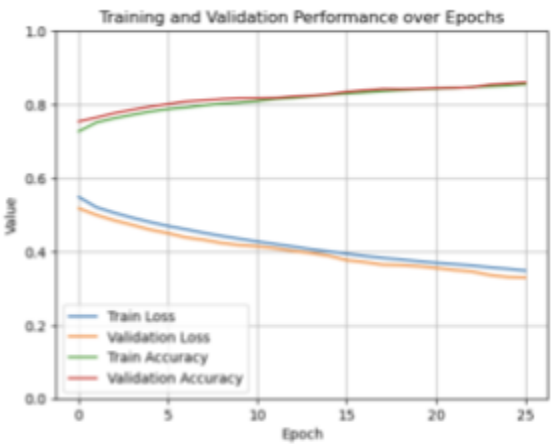


*Figure 4. Learning curve with LR = 0.001 over 30 epochs showing improved generalization and smoother convergence.*

In an effort to mitigate persistent overfitting signs, the learning rate was kept constant at 0.01 to isolate the impact of architectural changes, while two *nn.Dropout2d(p*=0.25) layers were added after the second pooling layer to improve generalization. The number of epochs was also reduced to 20 to further prevent overfitting observed in previous runs. This configuration led to a narrower gap between training and validation loss, with the two curves aligning more closely throughout training (see Figure 3). Final validation metrics were: Accuracy 89.3%, Precision = 93.1%, Recall = 80.6%, and F1-Score = 86.4%, reflecting a good overall performance.

In the final iteration, the learning rate was reduced from 0.01 to 0.001 to encourage smoother convergence, and the number of training epochs was increased to 30 to allow sufficient learning at the slower rate. This adjustment yielded a more stable and gradual decline in both training and validation loss, with noticeably reduced overfitting compared to previous runs. The performance metrics also reflect this balance: an accuracy of 86.9%, precision of 84.6%, recall of 84.1%, and F1-score of 84.3% on the validation set. Overall, this configuration demonstrated a great generalization behavior.

## 4. Final Results and Performance

After several tuning iterations, I selected the configuration in Figure 3 (LR = 0.01, Epochs = 20) as the final model version. Despite having slightly fewer epochs than earlier runs, this setup produced the best balance between generalization and performance: the validation loss remained stable, the training and validation accuracy curves aligned closely, and the metrics were strong, with an accuracy of 89.3%, precision of 93.1%, recall of 80.6%, and F1-score of 86.4% on the validation set.

One consistent observation throughout training was that the validation loss was often lower than the training loss (similar patterns observed for accuracy), which can initially appear counterintuitive. However, this behavior is usually expected when using Dropout, which is only active during training. When the model switches to evaluation mode (model.eval()), dropout is disabled, reducing the noise and typically resulting in a lower validation loss. There are more advanced strategies to further mitigate this behavior, such as early stopping, using dropout-like behavior during inference, or plotting additional metrics like precision, recall, and F1-score alongside loss. While I explored smaller learning rates (see Figure 4) to smooth training, I ultimately decided to stop here since the learning curve is stable and the final metrics reflect strong and consistent performance.

More tuning could still improve the results, but the current configuration provides a solid, interpretable baseline.

Confusion Matrix

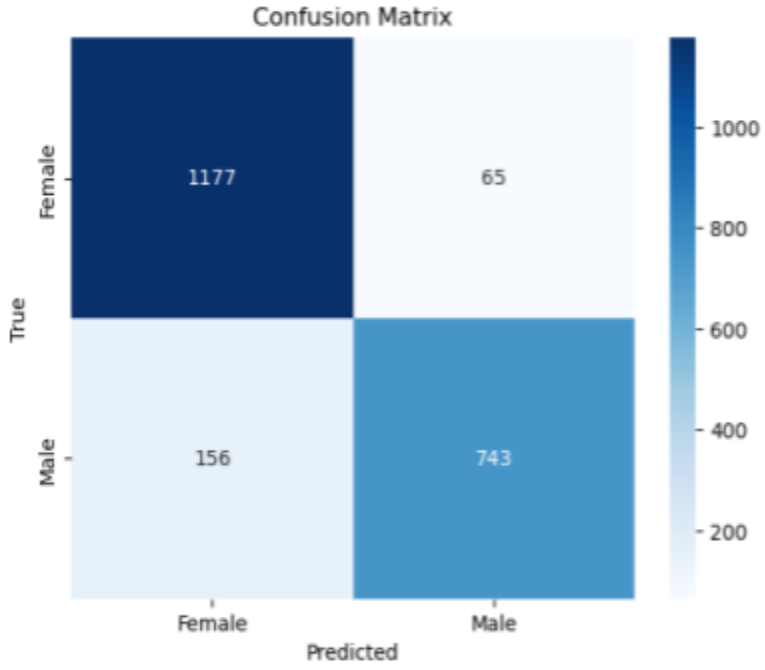


*Figure 5. Confusion matrix on validation set for the final model (LR = 0.01, Epochs = 20, Dropout = 0.25), showing accurate classification performance with slightly more false negatives in the male class.*

To gain a better understanding of the model's classification behavior, I computed the confusion matrix using the validation set predictions from the final epoch (Figure 5). While confusion matrices are typically computed on the test set, this was not possible due to the Kaggle challenge setup, where test labels are hidden.

The matrix reveals that the model correctly classified **1,177 out of 1,242 female samples** and **743 out of 899 male samples**. Most misclassifications occurred in predicting **males as females** (156 cases), indicating room for improvement in the model's ability to generalize across both classes. This suggests that the model may not be equally strong at identifying features associated with male and female images, highlighting the need for further tuning or data augmentation to improve balance..

Overall, the model demonstrates **strong discriminative performance** on both classes.

## 5. Conclusion

This project began as an early learning exercise in Deep Learning but was later revisited and improved to reflect more mature practices in training, evaluation, and model tuning. Through careful adjustments, including better data splitting, dropout regularization, learning rate tuning, and performance monitoring, I was able to significantly enhance the model's generalization and stability. The final model achieved **strong performance** across **accuracy, precision, recall, and F1-score**, supported by **a smooth learning curve** and a **well-behaved confusion matrix**. While further tuning could yield more gains, the current results represent a solid baseline for gender classification from X-ray images.